

PROPOSAL GAME ONLINE REAL-TIME BERBASIS WEBSOCKET
“TRUE OR FALSE BLITZ: REAL-TIME DECISION SPEED GAME”

Mata kuliah : Cloud Computing



Disusun Oleh:

Kelompok 6

Muhammad Adi Pratama Akasela	2410010388
M. Ridhoni Akbar	2410010376
Wira Yuda Pamuji	2410010396
Muhammad Yusuf	2410010526
Muhammad Abdi Nur Salam	2410010482
Muhammad Roby Hablirrohman	2410010418

Dosen Pengampu :

Fahtul Hafidh S.Kom., M.Kom.

FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ISLAM KALIMANTAN MUHAMMAD
ARYSAD AL-BANJARI
BANJARBARU
TAHUN 2026

DAFTAR ISI

KATA PENGANTAR	4
BAB I PENDAHULUAN.....	5
1.1 LATAR BELAKANG	5
1.2 Rumusan Masalah.....	6
1.3 TUJUAN	6
BAB II LANDASAN TEORI	8
2.1 Game Berbasis Web Real-Time	8
2.2 WebSocket.....	8
2.3 Arsitektur Sistem	8
BAB III METODE PENELITIAN	9
3.1 Rancangan Penelitian.....	9
3.2 Arsitektur Sistem	9
3.3 Sinkronasi	10
BAB IV HASIL PERANCANGAN SISTEM.....	11
4.1 Mekanisme Permainan.....	11
4.1.1 Alur Permainan	11
4.1.2 Aturan Main	12
4.2 Arsitektur Sistem	13
4.1.5 Diagram Arsitektur (client-server).....	13
4.2.1 Alur Komunikasi Real-Time.....	13
4.2.2 Mekanisme Sinkronisasi State	13
4.2.3 Port dan Protokol	13
4.3 Desain Database.....	14
4.2.4 Skema Tabel.....	14
4.2.5 Data untuk Analisis.....	14
BAB V DESAIN STATISTIK.....	15
5.1 Metrik Wajib.....	15
5.2 Metrik Tambahan.....	15
Perubahan performa antar ronde Membandingkan ronde awal dan akhir	15
5.3 Statistik	15
Accuracy = $\frac{\text{Jumlah Jawaban Benar}}{\text{Total Soal}} \times 100\%$	15
5.4 Contoh Simulasi Perhitungan Statistik	15

5.5 Analisis Performa Pemain Penggunaan Statistik dalam Analisis Performa Pemain	17
BAB VI IMPLEMENTASI DAN PENGUJIAN SISTEM	18
6.1 Implementasi Antarmuka Pengguna (User Interface)	18
a. Halaman Utama (<i>Main Menu</i>)	18
b. Halaman Papan Peringkat dan Statistik (<i>Leaderboard & Stats</i>).....	18
c. Halaman Gameplay Permainan.....	20
d. Kondisi Akhir Permainan (<i>Game Over Screen</i>)	21
BAB VII PENUTUP	23
7.1 Kesimpulan.....	23
7.2 Saran	23

KATA PENGANTAR

Puji syukur kami panjatkan kehadirat Allah SWT yang telah melimpahkan rahmat dan hidayah-Nya sehingga kami dapat menyelesaikan tugas besar mata kuliah Cloud Computing yang berjudul "True or False Blitz: Real-Time Decision Speed Game" ini dengan baik dan tepat waktu.

Proposal ini disusun sebagai bentuk perencanaan pengembangan game multiplayer berbasis WebSocket yang memanfaatkan teknologi cloud computing dalam implementasinya. Melalui proyek ini, kami berupaya mengaplikasikan konsep-konsep yang telah dipelajari selama perkuliahan, khususnya dalam bidang komunikasi real-time, arsitektur client-server, serta pengelolaan data berbasis web.

Kami menyadari bahwa dalam penyusunan proposal ini masih terdapat banyak kekurangan, baik dari segi teknis maupun penulisan. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari Bapak Fahtul Hafidh, S.Kom., M.Kom. selaku dosen pengampu mata kuliah Cloud Computing demi perbaikan ke depannya.

Akhir kata, kami berharap proposal ini dapat memberikan gambaran yang jelas mengenai rencana pengembangan sistem yang akan kami bangun, serta menjadi landasan yang kuat dalam pelaksanaan proyek tugas besar ini.

Banjarbaru, 5 Juli 2026

Kelompok 6
Tim Penyusun

BAB I

PENDAHULUAN

1.1 LATAR BELAKANG

Perkembangan teknologi web saat ini tidak lagi terbatas pada halaman yang bersifat statis, tetapi telah berkembang menjadi aplikasi yang interaktif, dinamis, dan mampu berkomunikasi secara real-time. Dalam bidang pendidikan, khususnya pada sistem kuis daring, masih banyak aplikasi yang menggunakan protokol HTTP dengan mekanisme request-response. Pada mekanisme ini, pengguna harus melakukan penyegaran halaman secara manual atau sistem harus melakukan pengecekan data secara berkala untuk memperoleh informasi terbaru. Cara tersebut sering menimbulkan keterlambatan pengiriman data, penggunaan bandwidth yang lebih besar, serta perbedaan waktu penerimaan informasi antar pengguna.

Pada lingkungan Cloud Computing, aplikasi yang melayani banyak pengguna secara bersamaan memerlukan sistem komunikasi yang cepat dan efisien. Salah satu teknologi yang dapat digunakan untuk memenuhi kebutuhan tersebut adalah WebSocket. Berbeda dengan HTTP biasa, WebSocket memungkinkan terjadinya komunikasi dua arah secara terus-menerus melalui satu koneksi TCP. Dengan demikian, server dapat mengirimkan informasi langsung kepada klien tanpa harus menunggu permintaan terlebih dahulu. Kemampuan ini sangat penting pada aplikasi permainan kuis berbasis waktu, seperti "True or False Blitz", karena kecepatan respons pemain berpengaruh langsung terhadap perolehan skor.

Meskipun demikian, pengembangan aplikasi real-time juga memiliki tantangan tersendiri, terutama pada pengelolaan data yang melibatkan banyak operasi baca dan tulis ke database. Jika setiap jawaban pemain langsung disimpan ke database secara terus-menerus, performa server dapat menurun akibat tingginya jumlah transaksi yang harus diproses dalam waktu singkat. Untuk mengatasi masalah tersebut, Kelompok 6 menerapkan arsitektur hibrida yang memisahkan proses permainan real-time dengan proses penyimpanan data.

Pada rancangan ini, Node.js dan Socket.IO digunakan untuk menangani komunikasi real-time serta pengelolaan status permainan, sedangkan proses penyimpanan data dilakukan melalui RESTful API berbasis PHP yang terhubung dengan database MySQL. Selain itu, data permainan tidak langsung disimpan setiap saat, melainkan dikumpulkan terlebih dahulu dan diproses pada akhir sesi permainan. Pendekatan ini memungkinkan sistem tetap menjaga konsistensi data sekaligus

mempertahankan performa aplikasi agar tetap responsif ketika melayani banyak pemain secara bersamaan.

1.2 Rumusan Masalah

1. Bagaimana merancang dan mengimplementasikan sistem komunikasi real-time berbasis WebSocket menggunakan Node.js dan library Socket.IO sehingga seluruh pemain dalam satu room dapat menerima pernyataan, mengirimkan jawaban, dan melihat pembaruan leaderboard secara bersamaan dan tersinkronisasi?
2. Bagaimana merancang mekanisme sinkronisasi state permainan di sisi server sebagai *single source of truth*, mencakup pengelolaan timer 8 detik, validasi jawaban, perhitungan skor beserta penalty, serta penanganan kondisi disconnect dan reconnect pemain secara otomatis menggunakan event-driven architecture pada Node.js?
3. Bagaimana merancang skema basis data relasional yang mampu mencatat seluruh data permainan secara akurat, termasuk tiga kolom timestamp (*response_start*, *response_end*, *response_restart*) untuk mendukung analisis waktu respons pemain secara empiris?
4. Bagaimana mengimplementasikan sistem klasifikasi jawaban berbasis waktu respons (*benar cepat*, *benar lambat*, *salah*, *tidak menjawab*) dan sistem rank berjenjang (*Pemula*, *Menengah*, *Mahir*, *Ahli*) yang dapat diperbarui secara otomatis setelah setiap sesi permainan selesai?
5. Apakah terdapat peningkatan akurasi dan penurunan waktu respons pemain secara signifikan seiring bertambahnya jumlah game yang dimainkan, serta apakah terdapat korelasi negatif antara kecepatan respons dan akurasi sebagaimana dihipotesiskan dalam H1, H2, dan H3

1.3 TUJUAN

1. Merancang dan mengimplementasikan sistem komunikasi real-time berbasis WebSocket menggunakan Node.js dan Socket.IO dengan 8 event kunci (*join_room*, *game_start*, *statement_send*, *answer_submit*, *answer_result*, *leaderboard*, *game_end*, *reconnect*) yang memungkinkan interaksi multiplayer secara serentak antara 2 hingga 8 pemain dalam satu room.
2. Membangun game server berbasis Node.js yang memanfaatkan arsitektur *event-driven* dan *non-blocking I/O* sebagai *single source of truth* untuk mengelola seluruh logika permainan, termasuk manajemen room, state

machine ronde, penalty engine (-25 poin untuk jawaban salah), mekanisme bonus kecepatan dan streak, serta penanganan disconnect dan reconnect secara otomatis.

3. Merancang dan mengimplementasikan skema basis data relasional dengan 8 tabel (*users*, *rooms*, *room_players*, *statements*, *game_sessions*, *rounds*, *answers*, *player_stats*) yang dilengkapi stored procedure untuk pembaruan rank dan statistik pemain secara otomatis pasca-game.
4. Mengimplementasikan sistem klasifikasi jawaban berbasis waktu respons dan sistem rank berjenjang yang hanya berlaku bagi pemain reguler (*status_id* = 1) dengan syarat minimum akumulasi poin dan win rate, serta mengukur performa pemain menggunakan 7 metrik statistik wajib (M-01 hingga M-07).
5. Menguji dan membuktikan tiga hipotesis penelitian, yaitu: (H1) peningkatan akurasi pemain minimal $10\% \pm 5\%$ dari game pertama ke game ke-10; (H2) penurunan rata-rata waktu respons minimal $15\% \pm 5\%$ setelah beberapa game; dan (H3) adanya korelasi negatif antara kecepatan respons dan akurasi sebesar lebih dari $30\% \pm 10\%$, berdasarkan data empiris yang dikumpulkan selama periode pengujian aplikasi.

BAB II

LANDASAN TEORI

2.1 Game Berbasis Web Real-Time

Game berbasis web real-time adalah permainan yang dijalankan melalui browser dan memungkinkan interaksi antar pemain terjadi secara langsung tanpa jeda yang berarti. Berbeda dengan aplikasi web konvensional yang bersifat request-response, game real-time membutuhkan koneksi yang persisten antara client dan server agar perubahan state permainan dapat langsung diterima oleh seluruh pemain secara bersamaan.

2.2 WebSocket

WebSocket adalah protokol komunikasi dua arah (full-duplex) yang berjalan di atas koneksi TCP. Berbeda dengan HTTP yang bersifat satu arah dan harus membuka koneksi baru setiap request, WebSocket mempertahankan koneksi yang terbuka secara terus-menerus sehingga server dapat mengirimkan data ke client kapan saja tanpa menunggu permintaan terlebih dahulu. Protokol ini sangat cocok digunakan untuk aplikasi yang membutuhkan pertukaran data secara cepat dan berkelanjutan seperti game multiplayer real-time.

2.3 Arsitektur Sistem

Arsitektur client-server adalah model komunikasi di mana client bertugas mengirimkan permintaan dan server bertugas memproses serta mengembalikan hasilnya. Dalam proyek ini arsitektur tersebut dikembangkan menjadi dua lapis, yaitu Node.js sebagai server utama yang menangani seluruh logika game secara real-time melalui WebSocket, dan PHP sebagai API pasif yang melayani permintaan data statistik dan leaderboard dari frontend melalui HTTP.

BAB III METODE PENELITIAN

3.1 Rancangan Penelitian

Penelitian ini menggunakan pendekatan perancangan sistem dalam mengembangkan game multiplayer real-time berbasis WebSocket dengan objek penelitian berupa game *True or False Blitz*. Pendekatan ini difokuskan pada perancangan dan implementasi sistem yang mampu mendukung interaksi antar pemain secara langsung (real-time). Tahapan penelitian meliputi:

1. **Analisis Kebutuhan Sistem:** Mengidentifikasi kebutuhan perangkat keras dan perangkat lunak, serta menganalisis kebutuhan fungsional seperti alur permainan, mekanisme jawaban benar/salah, sistem timer, dan kebutuhan komunikasi dua arah antar pemain.
2. **Perancangan Arsitektur:** Merancang struktur client-server dua lapis dengan Node.js sebagai server utama yang menangani seluruh logika game secara real-time melalui WebSocket, serta PHP sebagai API pasif untuk statistik dan leaderboard.
3. **Perancangan Gameplay:** Merumuskan aturan main, mekanisme timer 8 detik, sistem skor beserta bonus kecepatan, bonus streak, dan penalty jawaban salah, serta merancang antarmuka (*User Interface*) game yang responsif.
4. **Implementasi Sistem:** Menerjemahkan rancangan ke dalam kode program. Pada tahap ini dilakukan pembuatan server menggunakan Node.js dan Socket.IO, frontend menggunakan HTML, CSS, dan JavaScript, serta PHP API sebagai jembatan penyimpanan data ke database MySQL melalui HTTP POST.
5. **Pengujian (Testing):** Melakukan uji coba sistem secara langsung untuk memastikan sinkronisasi data antar pemain berjalan dengan baik, perhitungan skor dan penalty bekerja sesuai rumus yang dirancang, serta data jawaban tercatat dengan benar di tabel user dan response.

3.2 Arsitektur Sistem

Selama proses pengembangan, tim melakukan serangkaian iterasi untuk meningkatkan pengalaman pengguna dan kestabilan sistem. Beberapa pengembangan fitur yang diimplementasikan meliputi:

- **Optimalisasi Gameplay:** Penyesuaian durasi timer per soal, penerapan sistem klasifikasi jawaban berbasis waktu respons (*benar cepat, benar lambat, salah, tidak menjawab*), serta penyempurnaan mekanisme bonus streak agar lebih mendorong konsistensi pemain.
- **Fitur Statistik & Visual:** Integrasi PHP API untuk menyajikan data statistik dan leaderboard secara terpisah dari alur game, serta penggunaan Chart.js untuk visualisasi data akurasi dan waktu respons pemain.
- **Penyempurnaan Antarmuka:** Penyesuaian tampilan tombol TRUE/FALSE agar lebih responsif, penambahan animasi feedback jawaban (hijau untuk benar, merah untuk salah), serta tampilan leaderboard yang diperbarui secara real-time setiap ronde selesai.

3.3 Sinkronasi

Sistem menggunakan metode *server authoritative*:

1. Semua aksi pemain (jawaban, timer, skor) diproses sepenuhnya di Node.js
2. Node.js mengirimkan update hasil ke seluruh pemain dalam room secara bersamaan melalui Socket.IO
3. Client hanya bertugas menampilkan hasil yang diterima dari server, tanpa melakukan kalkulasi apapun secara mandiri

BAB IV

HASIL PERANCANGAN SISTEM

4.1 Mekanisme Permainan

True or False Blitz adalah game multiplayer berbasis real-time yang berfokus pada kecepatan pengambilan keputusan (decision speed). Pemain akan diberikan sebuah pernyataan dan harus menentukan apakah pernyataan tersebut benar (True) atau salah (False) dalam waktu terbatas.

Berbeda dengan kuis biasa, game ini menekankan keseimbangan antara kecepatan dan akurasi dalam pengambilan keputusan. Server akan mengirim pernyataan secara bersamaan ke semua pemain dalam satu room, dan setiap pemain harus merespon secepat mungkin untuk mendapatkan skor optimal.

4.1.1 Alur Permainan

Permainan dimulai ketika pemain bergabung ke dalam room yang tersedia melalui antarmuka game. Setelah semua pemain siap, server secara serentak mengirimkan sebuah pernyataan kepada seluruh pemain dalam room melalui mekanisme broadcast WebSocket. Bersamaan dengan itu, timer 8 detik langsung dimulai dari sisi server sebagai pengendali waktu yang bersifat mutlak.

Selama timer berjalan, setiap pemain harus segera menentukan apakah pernyataan yang ditampilkan bernilai True atau False dengan menekan tombol yang tersedia. Jawaban yang dikirimkan kemudian diproses langsung oleh server secara real-time, mencakup validasi kebenaran jawaban serta pencatatan waktu respons masing-masing pemain. Berdasarkan hasil pemrosesan tersebut, skor setiap pemain dihitung dengan mempertimbangkan dua faktor utama yaitu kebenaran jawaban dan kecepatan respons, di mana jawaban salah akan dikenakan penalty sedangkan jawaban benar mendapatkan skor dasar beserta bonus kecepatan dan bonus streak.

Setelah seluruh pemain menjawab atau timer habis, server memperbarui peringkat seluruh pemain dan mengirimkan data leaderboard terbaru ke semua client secara bersamaan. Proses ini kemudian berulang kembali dari awal untuk ronde berikutnya hingga seluruh ronde dalam satu sesi permainan selesai dimainkan.

4.1.2 Aturan Main

- a. 2 pemain dalam satu room
- b. Setiap ronde terdiri dari 1 pernyataan
- c. Pemain hanya dapat menjawab sekali per ronde
- d. Jawaban setelah waktu habis tidak dihitung
- e. Semua pemain menerima soal yang sama secara bersamaan
- f. Permainan berlangsung selama 15 ronde

4.1.3 Sistem Skor

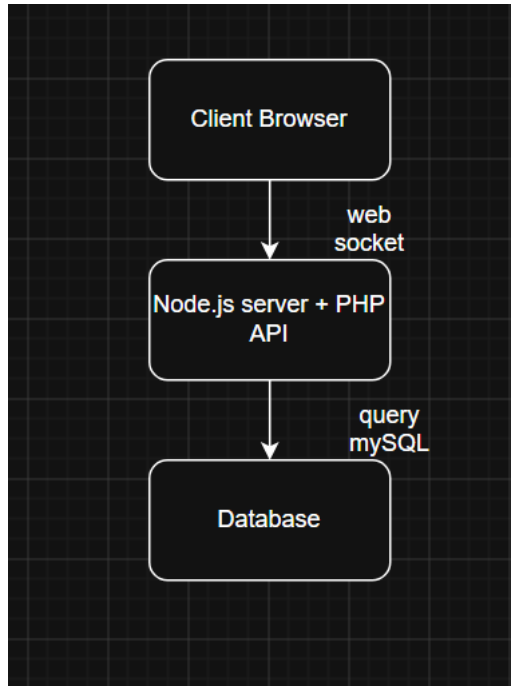
- a. Jawaban benar: +100 poin
- b. Bonus kecepatan:
 - < 2 detik: +50 poin
 - < 4 detik: +30 poin
 - < 6 detik: +10 poin
- c. Sistem penurunan skor berbasis waktu (time decay):
Semakin lama waktu respon, semakin kecil skor tambahan yang diperoleh
- d. Bonus streak:
 - 2 jawaban benar berturut-turut: +20 poin
 - ≥ 3 jawaban benar berturut-turut: +40 poin
- e. Jawaban salah: 0 poin

4.1.4 Kondisi Menang/Kalah

- a. Pemain dengan skor tertinggi setelah seluruh ronde selesai dinyatakan sebagai pemenang
- b. Ranking ditampilkan secara real-time setelah setiap ronde

4.2 Arsitektur Sistem

4.1.5 Diagram Arsitektur (client-server)



4.2.1 Alur Komunikasi Real-Time

- Player join room
- Server broadcast pernyataan
- Player mengirim jawaban
- Server mencatat waktu respon
- Server menghitung skor
- Server broadcast hasil dan ranking

4.2.2 Mekanisme Sinkronisasi State

- Timer dikontrol oleh server untuk menjaga fairness
- Semua pemain menerima event secara bersamaan
- Perhitungan skor dilakukan di server
- Client hanya menampilkan data

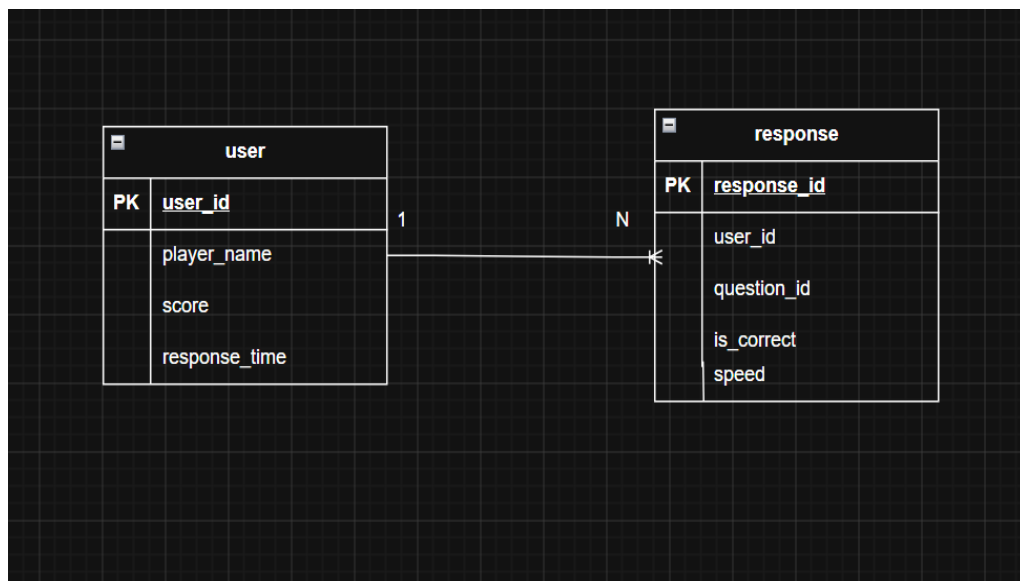
4.2.3 Port dan Protokol

Protokol	Port	Teknologi	Fungsi
WebSocket	3000	Node.js + Socket.IO	Komunikasi real-time seluruh event game

HTTP/HTTPS	80/443	Apache/Nginx	Serving frontend HTML, CSS, JS + PHP API
MySQL	3306	MySQL	Penyimpanan data user dan response

4.3 Desain Database

4.2.4 Skema Tabel



Relasi Antar Tabel

User → Response (1 : N)

Relasinya One to Many — satu pemain di tabel user dapat memiliki banyak baris jawaban di tabel response, karena setiap soal yang dijawab pemain dalam setiap ronde akan dicatat sebagai satu baris baru.

Satu ronde punya banyak jawaban dari pemain.

4.2.5 Data untuk Analisis

- Waktu respon tiap pemain
- Jumlah jawaban benar/salah
- Skor per ronde
- Riwayat permainan

BAB V

DESAIN STATISTIK

5.1 Metrik Wajib

a. Rata-rata waktu respon

Avg Time = total waktu respon / jumlah soal

b. Akurasi pemain

Accuracy = (jumlah jawaban benar / total soal) $\times$ 100%

c. Distribusi skor

Analisis persebaran skor antar pemain

5.2 Metrik Tambahan

a. Decision Speed Index (DSI)

Mengukur kombinasi kecepatan dan akurasi pemain

b. Streak Performance

Panjang rata-rata jawaban benar berturut-turut

c. Speed vs Accuracy Analysis

Analisis hubungan antara waktu respon dan akurasi

Perubahan performa antar ronde

Membandingkan ronde awal dan akhir

5.3 Statistik

a. Akurasi Jawaban (Accuracy)

Mengukur kecepatan pemain dalam menjawab 15 soal yang diberikan.

$$\text{Accuracy} = \left(\frac{\text{Jumlah Jawaban Benar}}{\text{Total Soal}} \right) \times 100\%$$

b. Rata-Rata Kecepatan Respon (Average Response Time)

Mengukur seberapa cepat pemain mengambil Keputusan (semakin kecil angkanya, semakin responsif).

$$\text{Avg Response Time} = \frac{\sum(\text{Waktu Respon Per Soal})}{\text{Total Soal yang Dijawab}} \text{ ms}$$

5.4 Contoh Simulasi Perhitungan Statistik

Sebagai ilustrasi, misalkan seorang pemain (A1) memainkan mode Singleplayer dengan total 15 soal. Hasil yang diperoleh adalah sebagai berikut:

- Jumlah jawaban benar: 12 soal

- Jumlah jawaban salah: 3 soal
- Bonus streak sebanyak 3 kali (tiga jawaban benar secara berurutan)
- Total waktu respons untuk seluruh soal: 30.000 ms

Berdasarkan data tersebut, dilakukan perhitungan sebagai berikut.

1. Perhitungan Akurasi

Akurasi dihitung menggunakan rumus:

$$\begin{aligned}\text{Accuracy} &= \frac{\text{Jumlah Jawaban Benar}}{\text{Total Soal}} \times 100\% \\ &= \frac{12}{15} \times 100\% = 80\%\end{aligned}$$

Dengan demikian, tingkat akurasi pemain adalah **80%**.

2. Perhitungan Rata-rata Waktu Respons

Rata-rata waktu respons diperoleh dengan membagi total waktu penyelesaian dengan jumlah soal.

$$\text{Average Response Time} = \frac{30.000 \text{ ms}}{15} = 2.000 \text{ ms}$$

Hasil tersebut menunjukkan bahwa pemain membutuhkan rata-rata **2 detik** untuk menjawab setiap soal.

3. Perhitungan Total Skor

Perhitungan skor dilakukan berdasarkan aturan permainan sebagai berikut:

- Skor jawaban benar:

$$12 \times 100 = 1.200 \text{ poin}$$

- Bonus streak:

$$3 \times 40 = 120 \text{ poin}$$

- Pengurangan akibat jawaban salah:

$$3 \times (-30) = -90 \text{ poin}$$

Sehingga total skor pemain adalah:

$$1.200 + 120 - 90 = 1.230 \text{ poin}$$

Berdasarkan hasil tersebut, data yang disimpan ke dalam basis data meliputi:

- **Total Score:** 1.230 poin
- **Average Response Time:** 2.000 ms
- **Accuracy:** 80%

5.5 Analisis Performa Pemain

Penggunaan Statistik dalam Analisis Performa Pemain

Statistik yang diperoleh dari hasil permainan dimanfaatkan sebagai dasar untuk mengevaluasi performa setiap pemain. Beberapa metrik yang digunakan meliputi tingkat akurasi, rata-rata waktu respons, dan total skor.

1. Mengukur Tingkat Penguasaan Materi

Nilai akurasi menunjukkan tingkat pemahaman pemain terhadap materi yang terdapat pada bank soal, seperti pengetahuan di bidang Teknologi Informasi dan Pengetahuan Umum. Sebagai contoh, akurasi sebesar **80%** menunjukkan bahwa pemain mampu menjawab sebagian besar soal dengan benar. Selain itu, rata-rata akurasi seluruh pemain juga dapat digunakan sebagai bahan evaluasi tingkat kesulitan soal. Apabila sebagian besar pemain memperoleh akurasi di bawah **50%**, maka soal dapat dikategorikan terlalu sulit dan perlu dilakukan penyesuaian.

2. Mengukur Kecepatan Berpikir dan Respons

Selain ketepatan jawaban, rata-rata waktu respons digunakan untuk mengukur kecepatan pemain dalam mengambil keputusan. Dua pemain dapat memperoleh akurasi yang sama, misalnya **100%**, tetapi pemain dengan rata-rata waktu respons yang lebih rendah, misalnya **1.200 ms** dibandingkan **3.500 ms**, menunjukkan kemampuan berpikir dan memahami soal yang lebih cepat. Oleh karena itu, metrik ini menjadi indikator tambahan dalam menilai performa pemain secara lebih objektif.

3. Evaluasi Kompetitif melalui Leaderboard

Kombinasi antara total skor, tingkat akurasi, dan kecepatan respons digunakan sebagai dasar penyusunan **leaderboard** permainan. Pemain yang mampu mempertahankan akurasi tinggi serta memberikan jawaban dalam waktu yang lebih singkat akan memperoleh skor yang lebih besar dan menempati peringkat yang lebih tinggi. Dengan demikian, sistem leaderboard dapat menciptakan kompetisi yang lebih adil sekaligus mendorong pemain untuk meningkatkan kemampuan dan konsistensi dalam bermain.

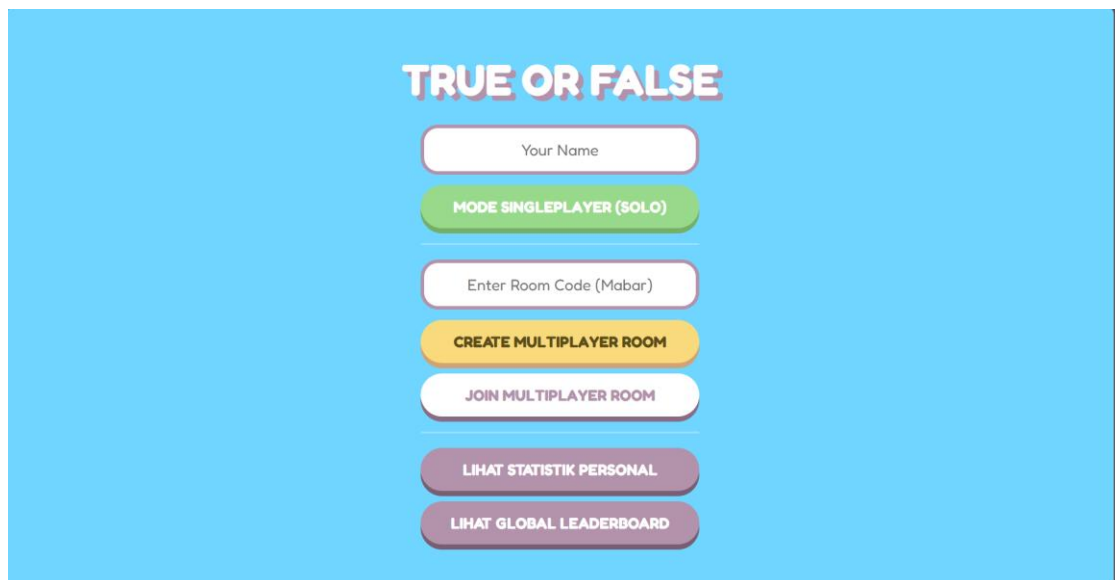
BAB VI

IMPLEMENTASI DAN PENGUJIAN SISTEM

6.1 Implementasi Antarmuka Pengguna (User Interface)

a. Halaman Utama (*Main Menu*)

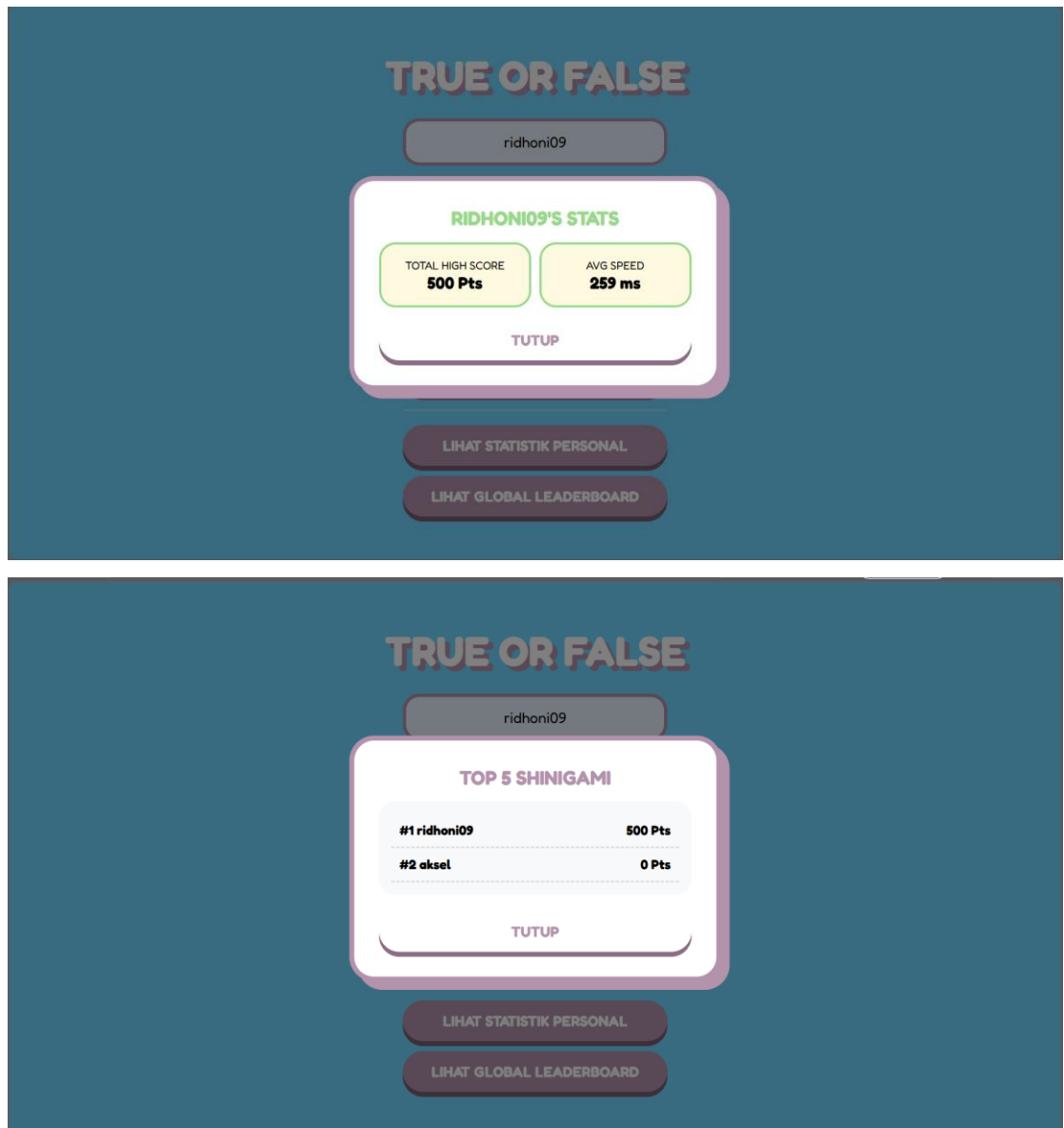
Halaman utama merupakan tampilan pertama yang muncul saat pengguna mengakses permainan. Desain antarmuka mengusung konsep visual yang ceria (*playful*) dengan menggunakan font Fredoka serta kombinasi warna pastel bertema *Monster-style*, seperti merah muda, kuning, biru, dan hijau. Pemilihan elemen visual tersebut bertujuan menciptakan kesan yang menarik dan ramah bagi pengguna sehingga pengalaman bermain menjadi lebih menyenangkan.



Dari sisi fungsional, halaman ini menyediakan kolom input teks (#name) yang wajib diisi oleh pemain sebagai identitas sebelum permainan dimulai. Setelah nama dimasukkan, pemain dapat memilih MODE SINGLEPLAYER (SOLO) untuk langsung memulai permainan mandiri yang terhubung ke server Node.js. Selain itu, tersedia kolom kode *room* beserta tombol CREATE dan JOIN MULTIPLAYER ROOM yang digunakan untuk membuat maupun bergabung ke ruang permainan secara *real-time*. Pada bagian bawah halaman juga disediakan tombol utilitas yang berfungsi membuka jendela pop-up statistik pribadi serta papan peringkat (*leaderboard*) global tanpa harus berpindah halaman.

b. Halaman Papan Peringkat dan Statistik (*Leaderboard & Stats*)

Halaman papan peringkat dan statistik ditampilkan menggunakan komponen Modal Overlay, yaitu jendela pop-up modern dengan latar belakang transparan gelap yang muncul di atas halaman utama. Pendekatan ini memungkinkan pengguna melihat informasi tambahan tanpa perlu melakukan pemuatan ulang (*reload*) halaman sehingga interaksi terasa lebih cepat dan nyaman.

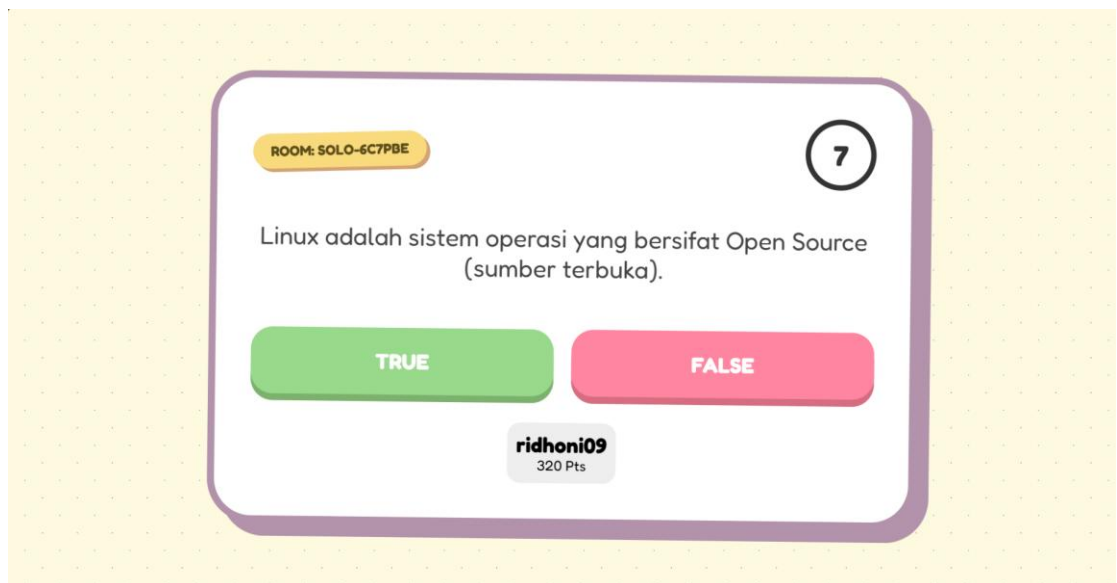
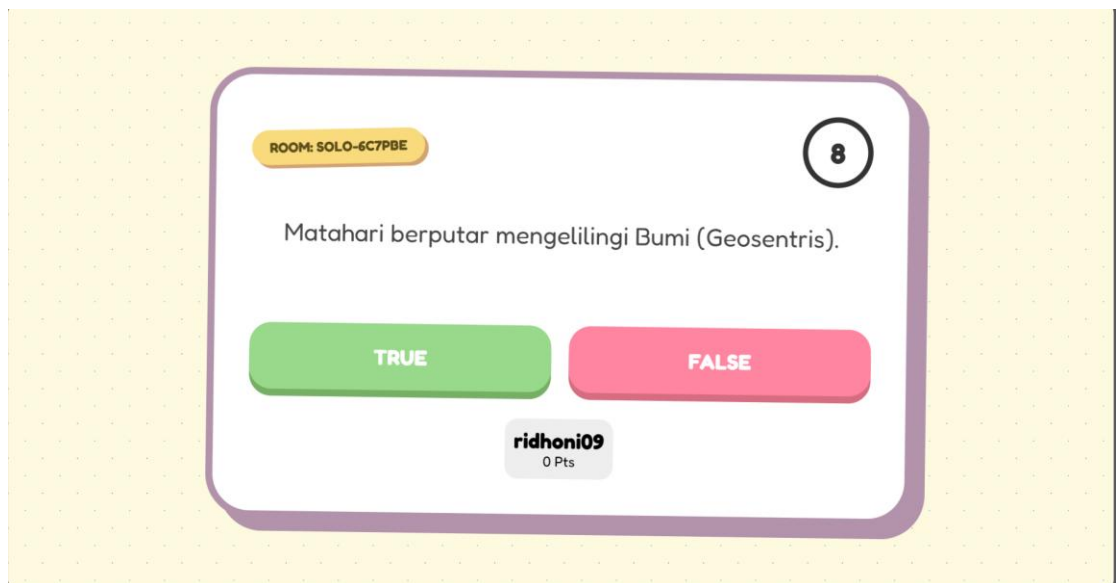


Menu statistik diakses melalui fungsi `viewStatsMenu()`. Setelah pemain memasukkan nama dan menekan tombol yang tersedia, sistem akan mengirim permintaan (*fetch*) ke API berbasis PHP untuk mengambil data dari basis data. Informasi yang ditampilkan meliputi total poin tertinggi yang pernah diperoleh serta rata-rata kecepatan respons pemain dalam satuan milidetik (ms). Sementara itu, menu Leaderboard dijalankan melalui fungsi

viewLeaderboardMenu(), yang secara otomatis mengambil data agregat dari database MySQL dan menampilkan lima pemain dengan skor tertinggi menggunakan label "TOP 5 SHINIGAMI", lengkap dengan total akumulasi poin yang diperoleh masing-masing pemain.

c. Halaman Gameplay Permainan

Saat permainan dimulai, tampilan menu utama secara otomatis disembunyikan menggunakan properti display: none, kemudian antarmuka permainan utama (.game) ditampilkan disertai animasi wobble agar suasana permainan terasa lebih dinamis. Perubahan tampilan ini memberikan transisi yang jelas dari menu menuju sesi permainan.



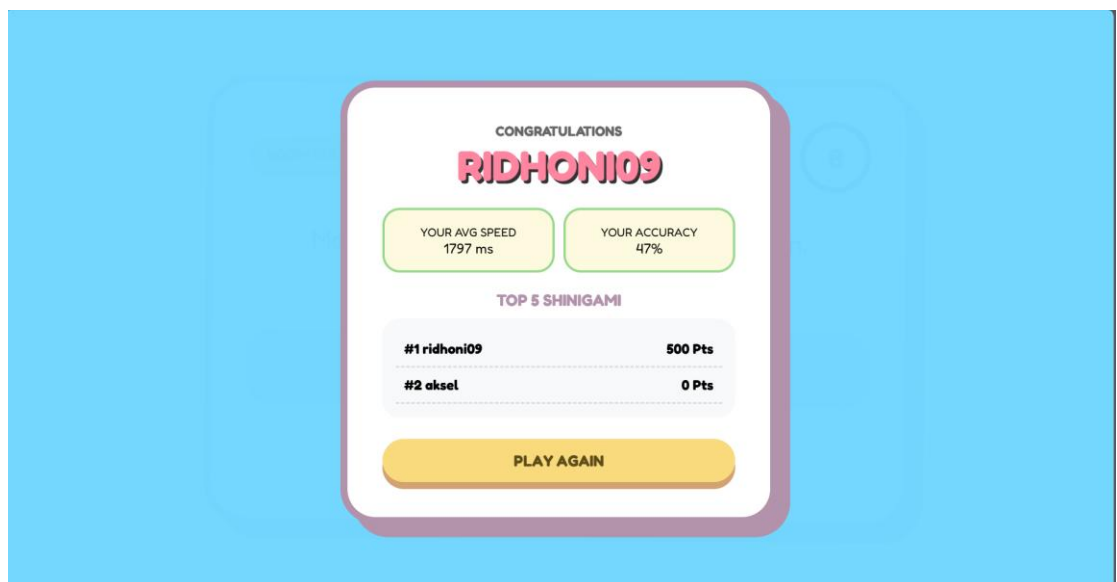
Pada bagian atas halaman terdapat informasi mengenai kode *room* yang sedang aktif beserta indikator penghitung waktu berbentuk lingkaran

(*countdown timer*) dengan durasi delapan detik untuk setiap soal. Ketika waktu yang tersisa mencapai tiga detik atau kurang, indikator timer akan berubah warna menjadi merah sebagai tanda bahwa waktu hampir habis. Di bagian tengah halaman ditampilkan pertanyaan pengetahuan umum maupun teknologi informasi yang dikirim secara dinamis oleh server Node.js. Di bawah pertanyaan tersedia dua tombol utama, yaitu tombol berwarna hijau untuk jawaban **TRUE** dan tombol berwarna merah muda untuk jawaban **FALSE**. Setelah salah satu tombol dipilih, sistem akan menonaktifkan (*disabled*) kedua tombol tersebut sehingga pemain tidak dapat mengubah maupun mengirim jawaban lebih dari satu kali.

Pada bagian bawah antarmuka permainan ditampilkan live scoreboard yang memperlihatkan daftar nama seluruh pemain di dalam *room* beserta perubahan skor mereka secara *real-time*. Dengan adanya papan skor ini, setiap pemain dapat memantau perkembangan pertandingan dan mengetahui posisi mereka dibandingkan pemain lain selama permainan berlangsung.

d. Kondisi Akhir Permainan (*Game Over Screen*)

Setelah seluruh pemain menyelesaikan soal ke-15, sistem akan menampilkan halaman akhir permainan berupa full-screen overlay berwarna biru cerah. Tampilan ini muncul secara otomatis sebagai penanda bahwa sesi permainan telah berakhir dan seluruh hasil pertandingan telah dihitung.



Pada halaman ini ditampilkan pengumuman pemenang utama menggunakan format huruf kapital, misalnya "A1 WIN". Selain itu, sistem menyajikan kartu statistik performa yang memuat informasi mengenai tingkat

akurasi jawaban dalam bentuk persentase serta rata-rata kecepatan respons pemain selama permainan berlangsung. Untuk mode Multiplayer, halaman ini juga menampilkan tabel hasil pertandingan (*Match Result*) yang berisi perbandingan skor akhir seluruh pemain di dalam *room*. Tabel tersebut tidak ditampilkan apabila permainan dilakukan dalam mode Singleplayer. Di bagian bawah halaman, sistem kembali memuat daftar "TOP 5 SHINIGAMI" yang telah diperbarui sehingga pemain dapat mengetahui apakah skor yang baru diperoleh berhasil masuk ke dalam lima peringkat tertinggi. Sebagai penutup, tersedia tombol PLAY AGAIN yang berfungsi untuk memuat ulang halaman permainan sehingga pengguna dapat langsung memulai sesi permainan berikutnya.

BAB VII

PENUTUP

7.1 Kesimpulan

Pengembangan sistem *True or False Blitz* menghasilkan beberapa konklusi pokok yang merangkum keseluruhan hasil dari proyek ini:

1. **Stabilitas Arsitektur *Real-Time*:** Penggunaan teknologi Node.js yang dikombinasikan dengan Socket.IO terbukti ampuh mengatasi kendala latensi. Sinkronisasi perhitungan waktu (*timer*), distribusi soal, dan pembaruan skor antar perangkat dalam satu *room* dapat tereksekusi secara instan tanpa hambatan berarti.
2. **Efektivitas *Gameplay*:** Konsep adu cepat memvalidasi pernyataan (benar/salah) berhasil menciptakan ritme permainan yang menuntut konsentrasi tinggi. Mekanika ini tidak hanya menyajikan hiburan kompetitif, tetapi juga memicu ketangkasan kognitif dan pengambilan keputusan di bawah tekanan.
3. **Manajemen Metrik Performa:** Sistem pencatatan statistik yang dibangun berjalan dengan sangat baik. Penggabungan *backend real-time* dengan penyimpanan *database* (melalui PHP/MySQL) sukses merekam data kompleks seperti rata-rata kecepatan respons dan persentase akurasi untuk disajikan pada fitur *Global Leaderboard*.

7.2 Saran

Berdasarkan hasil pengembangan dan pengujian sistem *True or False Blitz*, terdapat beberapa saran yang dapat diimplementasikan untuk pengembangan dan penyempurnaan proyek selanjutnya, khususnya dalam ruang lingkup *Cloud Computing*:

1. **Penerapan *Cloud Deployment (Hosting Publik)*:** Saat ini, arsitektur *game* masih dijalankan pada lingkungan server lokal (*localhost* / IP jaringan lokal). Untuk memaksimalkan implementasi konsep komputasi awan, disarankan agar *backend* Node.js maupun *database* PHP/MySQL di-*deploy* ke layanan *Virtual Private Server (VPS)* atau *Cloud Hosting* (seperti AWS, Google Cloud, atau layanan serupa). Dengan langkah ini, *game* dapat diakses oleh pemain dari berbagai perangkat secara global melalui internet publik.
2. **Skalabilitas Jaringan dengan Load Balancing:** Apabila *game* ini diproyeksikan untuk menampung jumlah pemain yang masif secara bersamaan (*high concurrent users*), infrastruktur Socket.IO pada Node.js dapat diperkuat

dengan menggunakan *Redis Adapter*. Hal ini akan mencegah terjadinya *bottleneck* pada memori server tunggal dan memungkinkan pembagian beban kerja (*load balancing*) ke beberapa *instance* server secara bersamaan.

3. **Peningkatan Keamanan dan Autentikasi Pengguna:** Sistem saat ini mengizinkan pengguna untuk langsung masuk dan bermain hanya dengan mengetikkan nama. Ke depannya, disarankan untuk mengimplementasikan sistem autentikasi terenkripsi (*login/register*). Tujuannya adalah untuk menjaga integritas data statistik di dalam *database*, sehingga skor tinggi milik seorang pemain tidak dapat diklaim atau ditimpa oleh pengguna lain.
4. **Pengayaan Fitur Kustomisasi *Room* secara Dinamis:** Untuk meningkatkan nilai guna aplikasi, pengembangan selanjutnya dapat menambahkan fitur pembuatan soal (Kustomisasi *Room*). Melalui fitur ini, pengguna atau *host* dapat menginput data soal ke dalam *database cloud* secara mandiri, sehingga aplikasi tidak hanya berfungsi sebagai sarana hiburan, tetapi juga dapat dimanfaatkan sebagai media kuis interaktif untuk keperluan akademis.